ARRAYS INDEXADOS E ASSOCIATIVOS

*O backup guardava as exclusões numa variável de texto. Funcionou meses, até alguém acrescentar `/home/kali/Meus Documentos` : o `rsync` recebeu `--exclude=/home/kali/Meus` e, à parte, `Documentos` — que virou **mais uma origem**. Pôr aspas piora: tudo vira um argumento gigante. **Cadeia de texto não guarda onde um item termina.***

060
GUIAS LINUX

AS QUATRO EXPANSÕES DE UM ARRAY INTEIRO

`arr=(um "dois com espaco" tres)` — a pergunta é sempre a mesma: o resultado vira **vários argumentos** ou **um texto**?

`"${arr[@]}"`	um · dois com espaco · tres	**3 palavras.** Um argumento por item, espaços preservados. **É a forma correta** para passar a comando, iterar num `for` ou copiar para outro array. Array vazio → zero argumentos.
`"${arr[*]}"`	um dois com espaco tres	**1 palavra.** Itens unidos pelo **primeiro caractere do** `IFS` . Só para exibir ou registrar em log. Outro separador: `( IFS=,; echo "${arr[*]}" )` num subshell.
`${arr[@]}`	um · dois · com · espaco · tres	**4 palavras.** Sem aspas, o `IFS` recorta de novo — o bug da abertura, agora com array.
`${arr[*]}`	um · dois · com · espaco · tres	**4 palavras**, idênticas às de cima: **sem aspas, `[@]` e `[*]` são indistinguíveis** — e ambos desfazem a estrutura que motivou o array.

`${#arr[@]}` conta **elementos** · `${#arr[0]}` conta **caracteres do primeiro** · `${!arr[@]}` lista os **índices existentes** · `${arr[-1]}` é o último · `${arr[@]:1:3}` fatia (o **espaço** em `${arr[@]: -2}` é obrigatório, senão o `:-` vira valor padrão). O par `@` / `*` é o mesmo de `"$@"` / `"$*"` — não é coincidência: os posicionais **são** um array.

A MESMA TABELA DE CHAVES — MUDA SÓ O TIPO DA CHAVE

`declare -a` · chave = inteiro · **esparso**

`[0]`	`a`	
`[2]`	`c`	índice 1 apagado
`[3]`	`d`	
`[9]`	`j`	saltou de 3 para 9

`${#arr[@]}` vale **4**, mas o maior índice é **9**. Um `for (( i=0; i<${#arr[@]}; i++ ))` perderia o `j` e leria dois índices vazios. `arr[100]=x` não aloca 97 posições — só acrescenta uma chave.

Percorra pelos valores ou pelos índices que existem: `for v in "${arr[@]}"` ou `for i in "${!arr[@]}"` . Reindexe com `arr=("${arr[@]}")` . Cite o `unset 'arr[1]'` — sem aspas o `[1]` pode virar glob.

`declare -A` · chave = texto qualquer

`[ssh]`	`22`	
`[http]`	`80`	
`[https]`	`443`	
`[dns rev]`	`53`	chave com espaço, citada

A ordem de `${!mapa[@]}` é a da tabela *hash*: **nem a de inserção, nem a alfabética**, e pode mudar entre versões do Bash. Relatório sai por `printf '%s\n' "${!mapa[@]}" | sort` .

Sem `declare -A` , o Bash trata como array indexado e avalia a chave como *aritmética*: `ssh` e `http` valem zero, tudo vai para o índice 0 sobrescrevendo-se — **e nenhum erro é emitido**. Dentro de função, `declare -A` cria variável local: use `declare -gA` para global.

CRIAR, CARREGAR E MONTAR A LINHA DE COMANDO

o padrão que resolve o problema da abertura em qualquer contexto

```
arr=(um dois tres)   arr+=(quatro)   arr[9]=dez   # literal, append, por indice
logs=( /var/log/*.log )               # do glob: cada arquivo e um item
mapfile -t linhas < arquivo.txt       # -t tira o \n; mais rapido e seguro que while read +=
mapfile -t users < <(cut -d: -f1 /etc/passwd)
IFS=: read -ra caminhos <<< "$PATH"   # de uma cadeia, com separador escolhido

# flags condicionais + caminho com espaco, sem eval e sem depender do IFS
argumentos=( -a --delete )
[[ -n ${VERBOSE:-} ]] && argumentos+=( -v )
[[ -n ${LIMITE:-}  ]] && argumentos+=( --bwlimit="$LIMITE" )
for dir in "${EXCLUSOES[@]}"; do argumentos+=( --exclude="$dir" ); done
$ printf 'rsync'; printf ' %q' "${argumentos[@]}"   # confira ANTES de executar
rsync -a --delete -v --bwlimit=2000 --exclude=/home/Meus\ Documentos
$ rsync "${argumentos[@]}" "$ORIGEM/" "$DESTINO/"
```

O que arrays não fazem: não aninham — o shell tem uma dimensão só (as saídas são chave composta `m["2,3"]` , ou aceitar que o problema pede `awk` ou Python) — e **não são passados nem devolvidos por funções**: expanda com `"${arr[@]}"` e reconstrua dentro (`local -a itens=( "$@" )`), ou use `local -n ref="$1"` quando a função

precisa *modificar* o array do chamador. E **arrays não são POSIX**: `arr=( ... )` falha no `dash` com `Syntax error: "(" unexpected` — se o script usa array, o *shebang* é `#!/usr/bin/env bash`, sem exceção.

RESUMO RÁPIDO

- ✓ **Lista de caminhos, alvos ou opções é array, nunca texto.** A cadeia depende do `IFS` e quebra no primeiro item com espaço; pôr aspas na cadeia só troca um defeito por outro.
- ✓ `"${arr[@]}"` **com aspas** produz um argumento por elemento — a forma certa em quase todo uso. `"${arr[*]}"` junta numa palavra só e serve para exibir. **Sem aspas, as duas viram a mesma coisa** e desfazem o array.
- ✓ `${#arr[@]}` **conta elementos;** `${#arr[0]}` **conta caracteres.** Os índices que existem estão em `${!arr[@]}`.
- ✓ **Arrays do shell são esparsos:** `unset` deixa buraco, e iterar por índice numérico pula elementos.
- ✓ **Associativo exige** `declare -A` **antes do primeiro uso** — sem isso a chave de texto vira aritmética, vale zero, e tudo se sobrescreve no índice 0 em silêncio.
- ✓ `mapfile -t` **carrega arquivo ou saída de comando** em array, uma linha por item — mais rápido e mais seguro que acumular com `+=` num `while read`.
- ✓ **Monte a linha com** `argumentos+=( … )` **e chame** `cmd "${argumentos[@]}"`**.** Resolve flag condicional e caminho com espaço sem `eval` — e o `eval` reintroduz injeção de comando sem resolver nada.
- ✓ Sob `set -u`, array vazio em `"${arr[@]}"` era fatal **até o Bash 4.3**; do 4.4 em diante (todo Kali atual) expande para nada, corretamente.

NO KALI

Lista de alvos autorizados é array, nunca cadeia de texto. `ALVOS=( 10.10.10.5 "servidor de teste.lab" )` sobrevive a nome com espaço e a lista vazia; `ALVOS="10.10.10.5 10.10.10.7"` depende do `IFS` e falha em silêncio no primeiro item atípico. Como o laço sobre alvos é justamente onde o filtro de escopo se aplica, o tipo de dado errado aqui vira **alvo varrido fora do escopo contratado** — crime pela Lei 12.737/2012. **Opções de** `nmap`**,** `gobuster` **e** `hydra` **montadas condicionalmente pedem array:** o *wrapper* que escolhe entre `-sS` e `-sT` conforme o privilégio, acrescenta `--max-rate` quando há limite contratado e insere o arquivo de saída só às vezes é exatamente o caso do `argumentos+=( … )` — com *string* e `eval`, uma senha ou um caminho com aspas na lista vira execução de comando. **Associativo é a estrutura natural para agregar resultados:** `achados["$ip:$porta"]="$banner"` dá deduplicação de graça e consulta direta, sem reprocessar arquivo — lembrando de ordenar na hora do relatório. `mapfile -t` **sobre arquivo de alvos é mais seguro que** `for alvo in $(cat alvos.txt)`, que divide por espaço e ainda expande glob numa linha de comentário; filtre com `[[ $alvo =~ ^[[:space:]]*(#|$) ]] && continue` dentro do laço. **Mas wordlist não cabe em array:** o `rockyou.txt` tem mais de 14 milhões de linhas e o `mapfile` carregaria tudo na memória do shell, com dezenas de bytes de sobrecarga por item. Wordlist se processa em fluxo com `while IFS= read -r`, ou vai direto para a ferramenta. **Array é para dezenas ou milhares de itens, não para milhões.**